

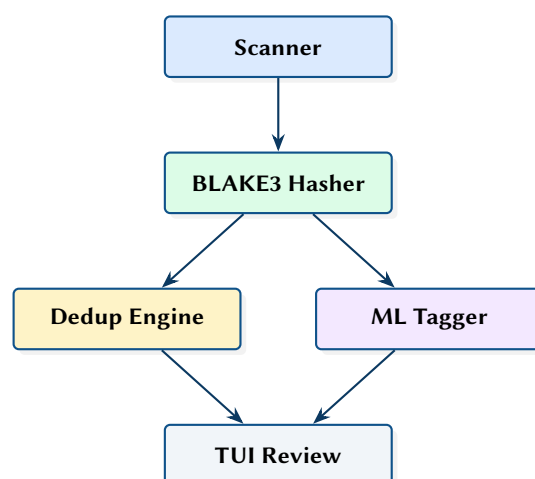
NAS Cleanup: High-Performance File Deduplication and ML-Powered Organisation for Synology NAS

BLAKE3 Hashing, BTRFS Reblink Dedup, and Neural Image Classification

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. This report presents a Rust application for managing large-scale file collections on Synology NAS systems. The tool traverses millions of files using parallel directory walking (jwalk), identifies bit-for-bit duplicates via BLAKE3 hashing with partial-hash pre-filtering, and deduplicates using BTRFS reflinks (copy-on-write) to reclaim space without deleting data. An ML classification pipeline using candle (MobileNetV3 and ResNet50) automatically categorises images and writes tags to XMP sidecar files. A ratatui-based TUI provides interactive review of duplicates and organisation plans. The tool is optimised for astrophotography and conventional photography workflows on Synology DSM.

Contents

1	Introduction and Motivation	2
2	Scanning and Hashing	2
2.1	Parallel Traversal	2
2.2	BLAKE3 Hashing	2
3	Duplicate Detection Pipeline	4
3.1	BTRFS Reblink Deduplication	4
4	ML Image Classification	5
5	TUI Review Interface	6
6	Design Summary	6
	List of Figures	7
	List of Tables	7
	List of Design Decisions & Rules	8
	Revision History	8
NAS	Network Attached Storage	2
BTRFS	B-tree File System	2
CoW	Copy-on-Write	2

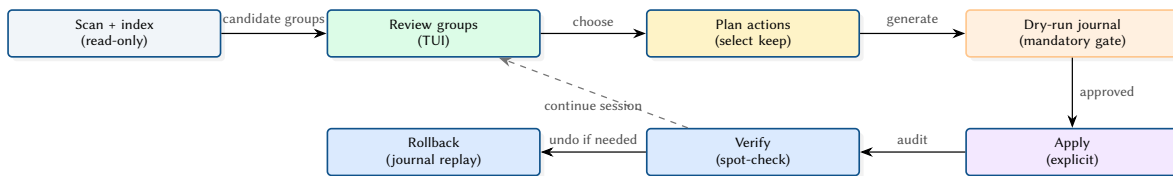
1 Introduction and Motivation

Large Network Attached Storage (NAS) systems accumulate duplicate files over years of use, particularly in photography workflows where RAW files are copied between editing workstations, imported from multiple cards, and backed up across volumes. Traditional deduplication tools are either too slow for million-file collections or require destructive deletion.

NAS Cleanup takes a different approach: it uses B-tree File System (BTRFS) reflinks to deduplicate at the block level, replacing duplicate files with Copy-on-Write (CoW) clones that share physical storage while maintaining independent file entries. This is non-destructive—each file retains its own path and metadata.

: Safety First

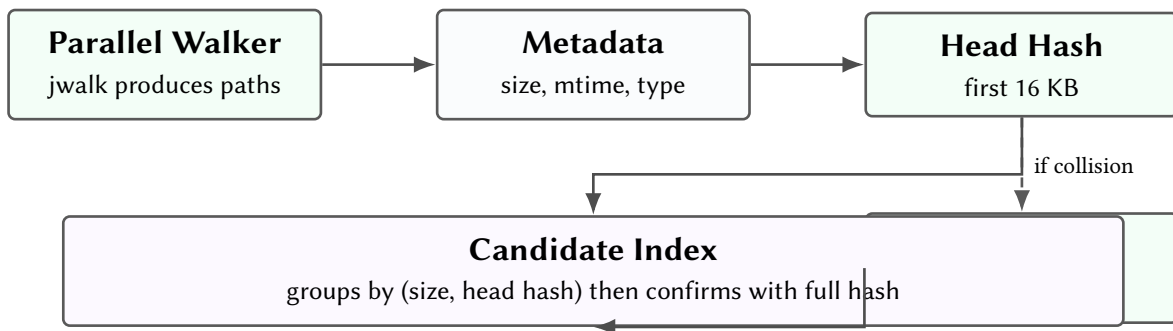
All destructive operations (delete, reflink, organise) support mandatory dry-run mode. The TUI defaults to read-only preview. The organise command generates a rollback journal that can undo all file moves.



Invariant: disk state changes only at **Apply**. All earlier states are deterministic and reproducible. The journal is the contract between human review and execution: it is both a dry-run report and a rollback recipe.

Figure 1. Journal/rollback semantics as a state machine. The critical safety property is that the tool never performs destructive actions directly from interactive review: it emits a journal (plan) that can be inspected, executed explicitly, and reversed.

2 Scanning and Hashing



I/O minimization. The head-hash stage is a cheap filter that avoids full reads for the majority of files. This keeps throughput bounded by sequential read bandwidth rather than random I/O across millions of small files.

Figure 2. Scanning and hashing pipeline with a two-phase hash strategy.

2.1 Parallel Traversal

Directory traversal uses jwalk [1] for parallel walking with Synology-specific exclusion patterns (@eaDir, #recycle, .DS_Store).

2.2 BLAKE3 Hashing

File hashing uses BLAKE3 [2] for SIMD-accelerated content hashing. A two-phase approach minimises disk I/O:

1. **Head hash** — Hash the first 16 KB of each file. Files with unique head hashes are immediately classified as unique.
2. **Full hash** — Only files with matching head hashes proceed to full-content hashing.

```
pub fn hash_file(path: &Path, partial: bool) -> Result<Hash> {
    let mut hasher = blake3::Hasher::new();
    let file = File::open(path)?;
    let mut reader = BufReader::with_capacity(65536, file);
    let limit = if partial { 16384 } else { usize::MAX };
    let mut total = 0;
    loop {
        let buf = reader.fill_buf()?;
        if buf.is_empty() || total >= limit { break; }
        let n = buf.len().min(limit - total);
        hasher.update(&buf[..n]);
        reader.consume(n);
        total += n;
    }
    Ok(hasher.finalize())
}
```

Candidate indexing structure

Progressive refinement: cheap keys first, expensive reads last

Key 0: size

Partition files by size; unique sizes are immediately unique

cheap

Key 1: head hash (16 KB)

Within each size bucket, compute partial hash; most buckets collapse to singletons

selective I/O

Key 2: full BLAKE3

Only for remaining collisions: full read + hash confirms exact duplicates

human-in-loop

Optional: perceptual hash / ML tags

Used for near-duplicate review and organization, not for auto-dedup

Insight:
The partial-hash optimisation reduces disk reads by 90%+ on typical photo collections where most files have unique first 16 KB (different EXIF timestamps, camera serial numbers, frame counters). Only genuine duplicates require full-file reads.

Figure 3. Algorithmic view of duplicate candidate indexing. The design goal is to keep the pipeline bounded by cheap metadata and short reads; only the small collision set triggers full file reads. This structure explains why the tool remains practical on million-file NAS volumes.

3 Duplicate Detection Pipeline

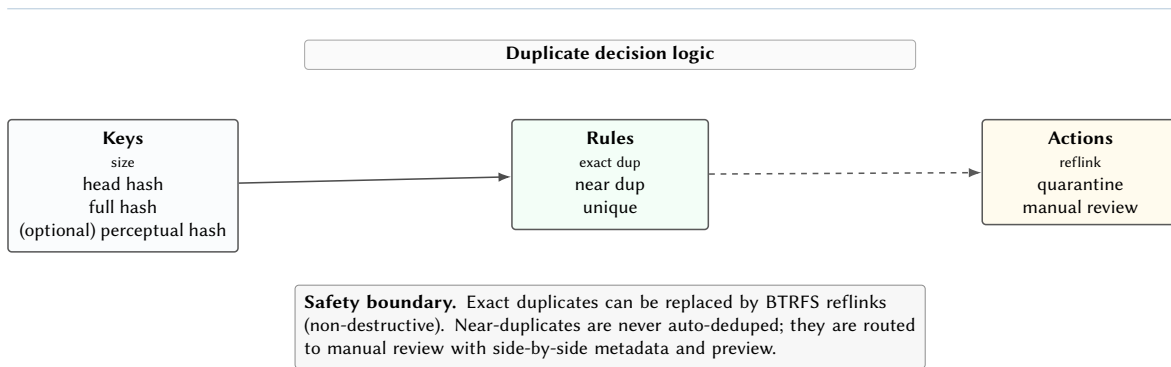


Figure 4. Duplicate detection as a decision system: evidence (hashes) is mapped to a conservative action set.

The four-stage pipeline progressively filters candidates:

1. **Discovery** — Parallel traversal builds file index
2. **Size filter** — Group by file size; unique sizes are immediately unique
3. **Head-hash filter** — 16 KB partial hash within size groups
4. **Full-hash verification** — Complete BLAKE3 hash for remaining candidates

Results are stored in a sled [3] embedded database for persistent indexing across sessions.

3.1 BTRFS Reblink Deduplication

On BTRFS volumes, duplicates are deduplicated using the `FICLONE` ioctl, which replaces file data with a [CoW](#) reference to the original. The `doctor` command verifies reflink support before operations.

Design Decision 1: Reblink vs Delete

Reblink deduplication is preferred over deletion because it preserves all file paths and metadata. Applications that reference files by path continue to work. The storage saving is equivalent to deletion, but the operation is reversible by simply copying (which triggers CoW, allocating new blocks).

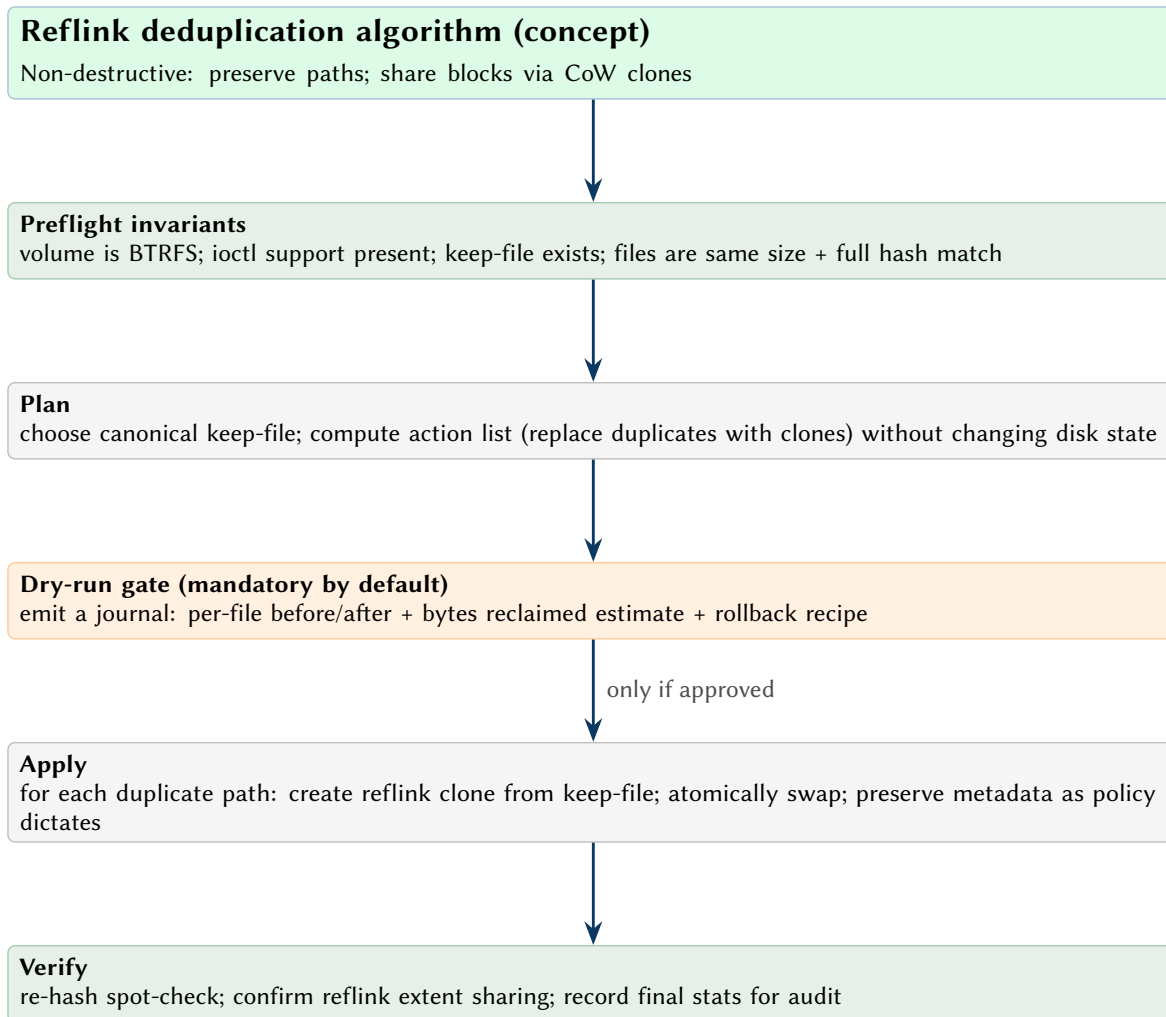


Figure 5. Reflink deduplication as a safety-gated algorithm. The key idea is that the system never "discovers then deletes"; it "proves duplicates, plans, dry-runs, then applies" with an auditable journal.

4 ML Image Classification

The ML pipeline uses candle [4] for inference:

Table 1. ML classification pipeline.

Pass	Model	Purpose
Fast scan	MobileNetV3	Basic categories (Landscape, Portrait, Night Sky, Architecture, etc.)
Detailed	ResNet50	Fine-grained taxonomy with confidence scores

Classification results are written to XMP sidecar files (.xmp) alongside the original images, preserving tags in a standard format readable by Lightroom, Capture One, and other photo editors.

5 TUI Review Interface

Insight:
Embedded JPEG extraction from RAW files (CR2, NEF, ARW, DNG) avoids the overhead of full RAW decoding for classification. The embedded preview is sufficient for scene classification and is typically 10–100× faster to extract than full demosaicing.

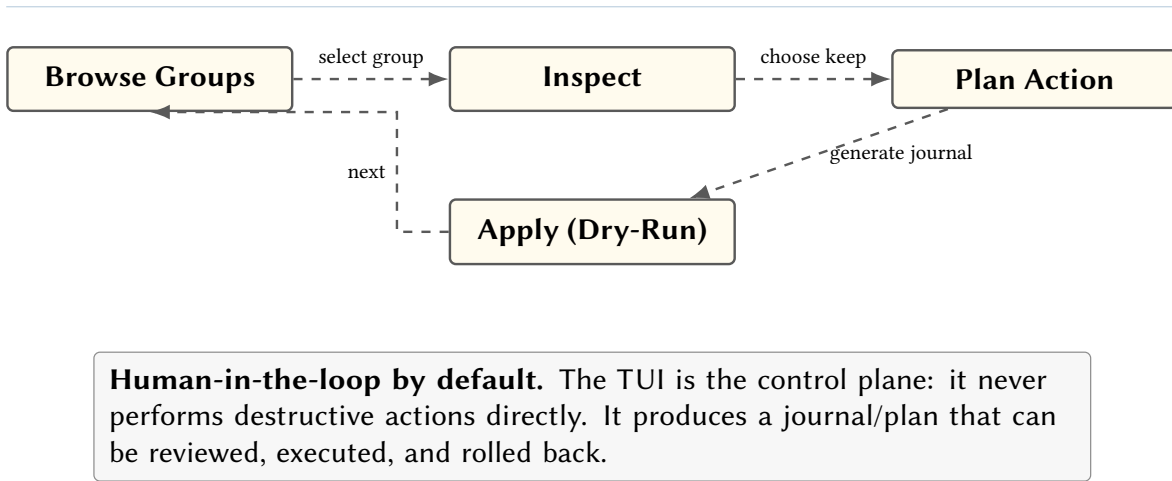


Figure 6. TUI review workflow state machine.

The ratatui [5] TUI provides interactive review of duplicates with:

- Side-by-side file metadata comparison
- Bulk selection with keyboard shortcuts
- Dry-run preview of all operations
- Real-time progress during scan and hash operations

6 Design Summary

Table 2. Key design decisions.

Decision	Rationale
BLAKE3	SIMD-accelerated; 3–5× faster than SHA-256
Partial hash pre-filter	90%+ disk read reduction on photo collections
BTRFS reflink	Non-destructive dedup; preserves all file paths
sled database	Persistent index across sessions; crash-safe
Two-pass ML	Fast MobileNetV3 triage; detailed ResNet50 only when needed
Embedded JPEG extraction	Avoids RAW decode overhead for classification

References

- [1] jwalk contributors, *jwalk: Blazing fast filesystem walk*, 2024. [Online]. Available: <https://crates.io/crates/jwalk>
- [2] J. O’Connor et al., *BLAKE3: One function, fast everywhere*, 2024. [Online]. Available: <https://github.com/BLAKE3-team/BLAKE3>
- [3] T. Neely, *Sled: An embedded database*, 2024. [Online]. Available: <https://sled.rs>
- [4] Hugging Face, *Candle: Minimalist ML framework for Rust*, 2024. [Online]. Available: <https://github.com/huggingface/candle>
- [5] Ratatui contributors, *Ratatui: A Rust library for cooking up TUIs*, 2024. [Online]. Available: <https://ratatui.rs>

List of Figures

1	Journal/rollback semantics as a state machine. The critical safety property is that the tool never performs destructive actions directly from interactive review: it emits a journal (plan) that can be inspected, executed explicitly, and reversed.	2
2	Scanning and hashing pipeline with a two-phase hash strategy.	2
3	Algorithmic view of duplicate candidate indexing. The design goal is to keep the pipeline bounded by cheap metadata and short reads; only the small collision set triggers full file reads. This structure explains why the tool remains practical on million-file NAS volumes.	3
4	Duplicate detection as a decision system: evidence (hashes) is mapped to a conservative action set.	4
5	Reflink deduplication as a safety-gated algorithm. The key idea is that the system never "discovers then deletes"; it "proves duplicates, plans, dry-runs, then applies" with an auditable journal.	5
6	TUI review workflow state machine.	6

List of Tables

1	ML classification pipeline.	5
2	Key design decisions.	6

List of Design Decisions & Rules

1	Design Rule: Safety First	2
1	Reflink vs Delete	4

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report